RICE UNIVERSITY

T2 · Kernels: run T1 on the GPU

Write two Triton kernels, a tiled matrix multiply and a fused cross-entropy, and train the same model on a GPU.

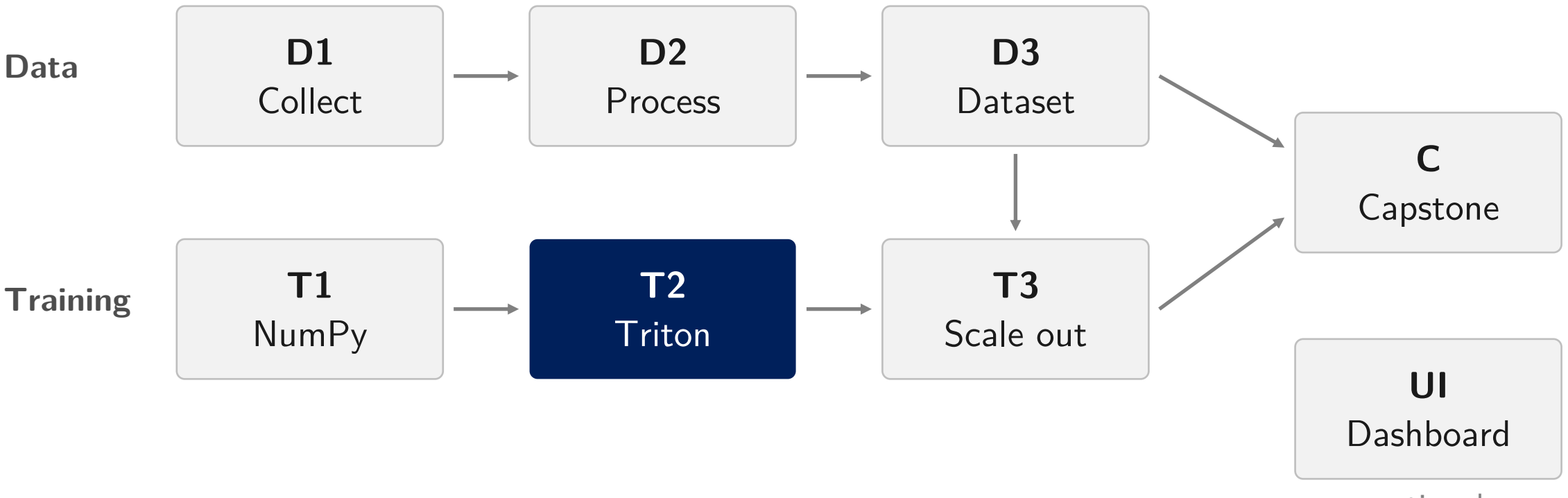
COMP 329 course staff

Rice University

Spring 2027 · Assignment T2

Release week 10 · Checkpoint week 11 · Final week 12 · 10-12 hours

Where this assignment fits



In our pipeline

This assignment: `runs/inputs/byte` → **T2** → `runs/t2-first`

What you will build, and why

Replace T1's most expensive operations with GPU kernels that give the same answers.

1 **Think in tiles**

A kernel is many small programs; each owns one block of the output.

2 **Edges matter**

Masks make odd shapes correct; strides make transposes correct.

3 **Measure honestly**

GPU work is asynchronous, and a tiny model may run faster on the CPU.

What goes in, what comes out

T1 model + byte shards
`runs/inputs/byte`
T1 code is provided

→

T2
Triton

→

GPU-trained model
`runs/t2-first`
Must match the NumPy run

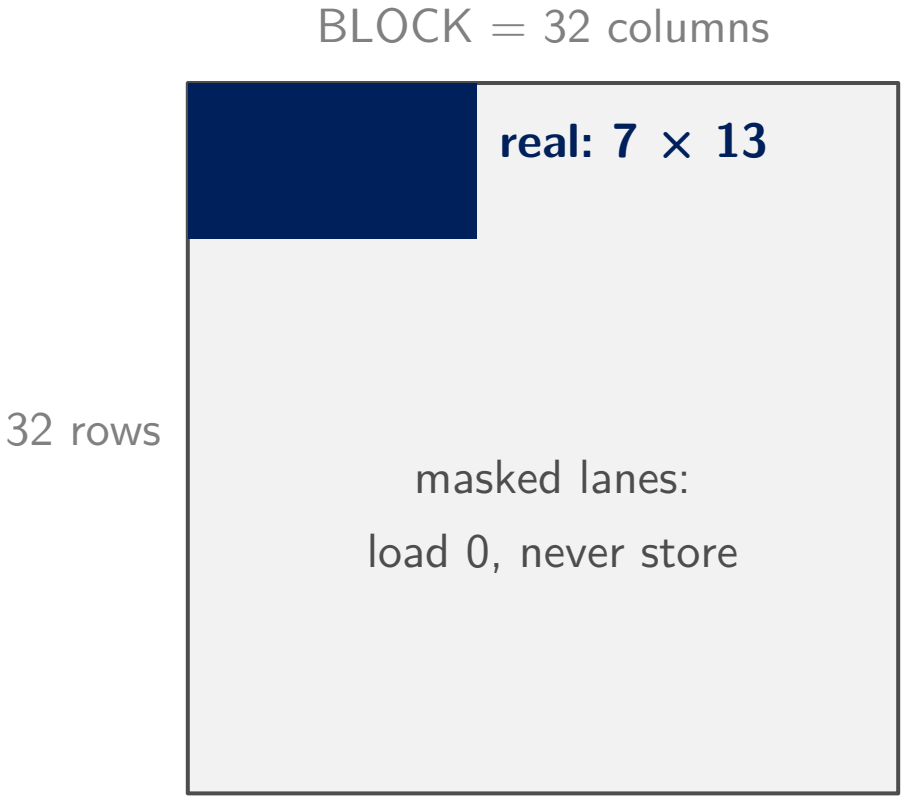
Files you produce

`matmul_kernel`
One output tile per program, float32 IEEE dots

`cross_entropy_kernel`
One row per program, loss and gradient fused

`Training run`
Same settings as T1, compared with the CPU run

One program computes one masked tile



(7, 19) @ (19, 13): every dimension has an edge.

1 **Tile offsets**

```
rows = pid_m·32 + arange(32)
cols = pid_n·32 + arange(32)
```

2 **Masked loads**

```
mask r < M, k < K; other = 0
```

3 **Accumulate in float32**

```
acc += tl.dot(a, b,
  input_precision="ieee")
```

4 **Masked store**

```
store only rows < M, cols < N
```

Fused, stable cross-entropy

```
row = tl.program_id(0)
cols = tl.arange(0, BLOCK)    # 512
mask = cols < V               # V = 322
x = tl.load(ptr + cols, mask,
            other=-inf)
m = tl.max(x)
# loss = log Σ exp(x - m) + m - x[y]
# grad = (p - onehot) / N
```

1 **One program per example**

N = batch size; V = real vocabulary.

2 **Pad with $-\infty$**

Lanes 322–511: $\exp(-\infty) = 0$, so they never act like a token.

3 **Same math as T1**

Row max and log-normalizer; divide by N once.

4 **Why fuse?**

No intermediate probability array moves between kernels.

The four Triton operations you need

`tl.program_id(axis)`
`Which program am I?`
Here: which output tile.

`tl.arange(0, B)`
`Lane offsets inside one block.`
Combine with strides for pointers.

`tl.load(p, mask, other)`
`Read valid lanes only.`
Use 0 for matmul operands.

`tl.store(p, v, mask)`
`Write valid lanes only.`
Never write outside the matrix.

Key idea

Strides are in elements: $A[r, k] = A + r \cdot AM + k \cdot AK$. A transposed array is just a view with different strides.

What you write, what you get

You write

`matmul_kernel`

`starter/kernels.py` · 3 numbered steps

`cross_entropy_kernel`

`starter/kernels.py`

Provided

- CuPy Tensor wrapper and allocation
- Launch grids and strides
- T1 model and training (provided/)
- Public tests in `tests/test_t2.py`

Ground rules

Keep the signatures · don't edit `starter/pipeline_lab` · run through `course.py` or `lab.py`.

Build it in this order

1 2 3 4 5

Learn the ops

`program_id`, arange, masked load and store.

Matmul

Tile offsets, masked loads, float32 accumulate.

Cross-entropy

Pad with $-\infty$, fuse loss and gradient.

Train on GPU

Same settings as the CPU run.

Measure

Synchronize timers; JIT reported separately.

```
check t2 --checkpoint --systems
```

```
check t2 --systems
```

```
lab.py t2
```

Check

Done when: All 4 tests pass on a GPU, with no skipped hardware tests.

If something goes wrong

Only odd shapes fail
Check: Masks for M, N and the final K tile.

Only transposed inputs fail
Check: Use element strides; don't assume row-major.

NaN loss when V = 322
Check: Pad with $-\infty$ and use stable reductions.

Gradient too large by N
Check: The mean loss divides by N once.

Runtime looks like zero
Check: Synchronize the GPU before and after timing.

Training hits a T1 TODO
Check: Use the provided T1 or finish yours; kernel tests are separate.

The performance study

Does this tiny workload pay for GPU launch cost?

Predict first

Predict before timing. Expect the GPU to lose on small matrices.

Kernel timings
checkpoint shapes + larger shapes

40-step training
CPU vs. GPU, same model and data

How to measure: One warmup (compile excluded) + three repeats. Synchronize around the timer; report setup/JIT time separately.

How you are graded

20	40	16	24
Checkpoint	Final correctness	Performance	Analysis

Correctness: GEMM 18 · fused loss/gradient 12 · training equivalence 10

Check

Done when: All 4 tests pass on a GPU, with no skipped hardware tests.

Your first ten minutes

```
$ bash setup.sh
$ source .venv/bin/activate
$ python course.py preflight
$ python course.py inputs --output runs/inputs
$ python course.py check t2 --checkpoint \
    --systems
$ python lab.py t2 --input runs/inputs/byte \
    --output runs/t2-first --steps 40
```

Your folder

`05_t2_triton/`

Handout:
`05_t2_triton/README.md`